

TASK 1

```
In [2]: import numpy as np
import pandas as pd

X = pd.read_csv("/Users/qayyumkhan/Downloads/kmeans_data/data.csv", header=None).values
y = pd.read_csv("/Users/qayyumkhan/Downloads/kmeans_data/label.csv", header=None).values.flatten()

print(X.shape, y.shape)

(10000, 784) (10000,)
```

Q1

```
In [4]: import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, normalize

# 1) Euclidean: z-score features
scaler_eu = StandardScaler(with_mean=True, with_std=True)
X_eu = scaler_eu.fit_transform(X)

# 2) Cosine: L2 normalize each sample (row)
X_cos = normalize(X.astype(float), norm='l2', axis=1)

# 3) Generalized Jaccard: ensure non-negative + scale to [0,1]
scaler_jac = MinMaxScaler(feature_range=(0, 1))
X_jac = scaler_jac.fit_transform(X.astype(float))

print("Shapes:", X_eu.shape, X_cos.shape, X_jac.shape)
print("Sanity checks:")
print("  Euclidean z-score mean≈", np.mean(X_eu), "std≈", np.std(X_eu))
print("  Cosine row norms (first 5):", np.linalg.norm(X_cos[:5], axis=1))
print("  Jaccard range min/max:", X_jac.min(), X_jac.max())
```

Shapes: (10000, 784) (10000, 784) (10000, 784)

Sanity checks:

Euclidean z-score mean $\approx$ -2.0391851472706956e-19 std $\approx$ 0.9230605702371493

Cosine row norms (first 5): [1. 1. 1. 1. 1.]

Jaccard range min/max: 0.0 1.0

```
In [5]: import numpy as np

# distance functions
def euclidean_distance(a, b):
    return np.linalg.norm(a - b)

def cosine_distance(a, b):
    denom = (np.linalg.norm(a) * np.linalg.norm(b)) + 1e-12
    return 1.0 - (np.dot(a, b) / denom)

def jaccard_distance(a, b):
    num = np.minimum(a, b).sum()
    den = np.maximum(a, b).sum() + 1e-12
    return 1.0 - (num / den)

# reusable K-means
def kmeans(X, K, distance_func, max_iters=500, random_state=42, stop_rule="centroid"):
    """
    stop_rule in {"centroid", "sse_increase", "max_only"}:
    - centroid: stop when centroids stop changing
    - sse_increase: stop if SSE increases on next iteration
    - max_only: only stop at max_iters
    """
    rng = np.random.default_rng(random_state)
    n, d = X.shape
    centroids = X[rng.choice(n, K, replace=False)].copy()

    def assign_labels(X, centroids):
        labels = np.empty(n, dtype=int)
        for i, x in enumerate(X):
            dists = [distance_func(x, c) for c in centroids]
            labels[i] = int(np.argmin(dists))
        return labels
```

```
def recompute_centroids(X, labels, K):
    new_c = np.zeros_like(centroids)
    for k in range(K):
        mask = labels == k
        if mask.any():
            new_c[k] = X[mask].mean(axis=0)
        else:
            new_c[k] = X[rng.integers(0, n)]
    return new_c

def compute_SSE(X, labels, centroids):
    sse = 0.0
    for i, x in enumerate(X):
        c = centroids[labels[i]]
        d = distance_func(x, c)
        sse += d ** 2
    return float(sse)

prev_sse = np.inf
for it in range(1, max_iters + 1):
    labels = assign_labels(X, centroids)
    sse = compute_SSE(X, labels, centroids)
    new_centroids = recompute_centroids(X, labels, K)

    stop = False
    if stop_rule == "centroid":
        if np.allclose(new_centroids, centroids):
            stop = True
    elif stop_rule == "sse_increase":
        if sse > prev_sse:
            stop = True
    elif stop_rule == "max_only":
        stop = False

    centroids = new_centroids
    if stop:
        print(f"Converged in {it} iterations.")
        return labels, centroids, sse, it
```

```

    prev_sse = sse

    labels = assign_labels(X, centroids)
    sse = compute_SSE(X, labels, centroids)
    print(f"Reached max_iters={max_iters}.")
    return labels, centroids, sse, max_iters

print("K-means + distances are ready.")

```

K-means + distances are ready.

```

In [6]: K = 10

# Euclidean
lbl_eu, cen_eu, sse_eu, it_eu = kmeans(X_eu, K, euclidean_distance, stop_rule="centroid")

# Cosine
lbl_co, cen_co, sse_co, it_co = kmeans(X_cos, K, cosine_distance, stop_rule="centroid")

# Jaccard
lbl_ja, cen_ja, sse_ja, it_ja = kmeans(X_jac, K, jaccard_distance, stop_rule="centroid")

print("SSE (Euclidean):", sse_eu, " | iters:", it_eu)
print("SSE (Cosine):   ", sse_co, " | iters:", it_co)
print("SSE (Jaccard):  ", sse_ja, " | iters:", it_ja)

# Which is better (lowest SSE)?
methods = [("Euclidean", sse_eu), ("Cosine", sse_co), ("Jaccard", sse_ja)]
best = min(methods, key=lambda t: t[1])
print("\nBest (by SSE):", best[0])

```

Converged in 56 iterations.

Converged in 34 iterations.

Converged in 55 iterations.

SSE (Euclidean): 5570520.866932062 | iters: 56

SSE (Cosine): 686.4221175146964 | iters: 34

SSE (Jaccard): 3665.691235759492 | iters: 55

Best (by SSE): Cosine

So, Cosine-K-means is the best because it gives the lowest SSE, meaning the clusters are the most compact under its distance measure.

In [8]: # Q2

```
In [9]: import numpy as np
from collections import Counter

def majority_vote_cluster_labels(cluster_labels, true_labels, K):
    """
    For each cluster k, find the majority true label inside that cluster.
    Returns:
        cluster_to_class: dict {cluster_id - predicted_class}
        y_pred: predicted labels for every data point
    Ties break by choosing the smallest label.
    """
    cluster_to_class = {}

    for k in range(K):
        idx = np.where(cluster_labels == k)[0]
        if len(idx) == 0:
            cluster_to_class[k] = 0
            continue

        labels_in_cluster = true_labels[idx]
        counts = Counter(labels_in_cluster)
        max_count = max(counts.values())
        majority_classes = [c for c, v in counts.items() if v == max_count]
        cluster_to_class[k] = min(majority_classes) # tie-break

    # assignning predicted class to each sample
    y_pred = np.array([cluster_to_class[c] for c in cluster_labels])
    return cluster_to_class, y_pred

def clustering_accuracy(cluster_labels, true_labels, K):
    _, y_pred = majority_vote_cluster_labels(cluster_labels, true_labels, K)
    acc = (y_pred == true_labels).mean()
```

```

    return acc

K = 10

acc_eu = clustering_accuracy(lbl_eu, y, K)
acc_co = clustering_accuracy(lbl_co, y, K)
acc_ja = clustering_accuracy(lbl_ja, y, K)

print("Accuracy (Euclidean-Kmeans):", acc_eu)
print("Accuracy (Cosine-Kmeans): ", acc_co)
print("Accuracy (Jaccard-Kmeans): ", acc_ja)

best_acc = max([("Euclidean", acc_eu), ("Cosine", acc_co), ("Jaccard", acc_ja)], key=lambda t: t[1])
print("\nBest (by accuracy):", best_acc[0])

```

Accuracy (Euclidean-Kmeans): 0.5006

Accuracy (Cosine-Kmeans): 0.6322

Accuracy (Jaccard-Kmeans): 0.5989

Best (by accuracy): Cosine

So Cosine-K-means is better, because it gives the highest accuracy. Intuitively, cosine distance focuses on the pattern of the 784-dim feature vectors (like digit shapes), not their raw magnitude, so it separates classes more cleanly than Euclidean or Jaccard on this dataset.

In [11]: # Q3

```

In [12]: import time

def kmeans_or_stop(X, K, distance_func, max_iters=500, random_state=42):
    rng = np.random.default_rng(random_state)
    n, d = X.shape
    centroids = X[rng.choice(n, K, replace=False)].copy()

    def assign_labels(X, centroids):
        labels = np.empty(n, dtype=int)
        for i, x in enumerate(X):

```

```

        dists = [distance_func(x, c) for c in centroids]
        labels[i] = int(np.argmin(dists))
    return labels

def recompute_centroids(X, labels, K):
    new_c = np.zeros_like(centroids)
    for k in range(K):
        mask = labels == k
        if mask.any():
            new_c[k] = X[mask].mean(axis=0)
        else:
            new_c[k] = X[rng.integers(0, n)]
    return new_c

def compute_SSE(X, labels, centroids):
    sse = 0.0
    for i, x in enumerate(X):
        c = centroids[labels[i]]
        d = distance_func(x, c)
        sse += d ** 2
    return float(sse)

prev_sse = np.inf

start = time.perf_counter()
for it in range(1, max_iters + 1):
    labels = assign_labels(X, centroids)
    sse = compute_SSE(X, labels, centroids)
    new_centroids = recompute_centroids(X, labels, K)

    # OR stop criteria:
    no_centroid_change = np.allclose(new_centroids, centroids)
    sse_increased = sse > prev_sse

    if no_centroid_change or sse_increased:
        end = time.perf_counter()
        return labels, new_centroids, sse, it, (end - start), no_centroid_change, sse_increased

centroids = new_centroids

```

```

        prev_sse = sse

    end = time.perf_counter()
    return labels, centroids, sse, max_iters, (end - start), False, False

K = 10
max_iters = 500

# Euclidean
_, _, sse_eu3, it_eu3, t_eu3, stop_c_eu, stop_s_eu = kmeans_or_stop(X_eu, K, euclidean_distance, max_

# Cosine
_, _, sse_co3, it_co3, t_co3, stop_c_co, stop_s_co = kmeans_or_stop(X_cos, K, cosine_distance, max_

# Jaccard
_, _, sse_ja3, it_ja3, t_ja3, stop_c_ja, stop_s_ja = kmeans_or_stop(X_jac, K, jaccard_distance, max_

print("Euclidean -> iters:", it_eu3, "time(s):", t_eu3, "stop_centroid:", stop_c_eu, "stop_sse_incr
print("Cosine     -> iters:", it_co3, "time(s):", t_co3, "stop_centroid:", stop_c_co, "stop_sse_incr
print("Jaccard    -> iters:", it_ja3, "time(s):", t_ja3, "stop_centroid:", stop_c_ja, "stop_sse_incr

```

```

Euclidean -> iters: 56 time(s): 12.452751875000104 stop_centroid: True stop_sse_increase: False
Cosine     -> iters: 34 time(s): 15.167728165997687 stop_centroid: True stop_sse_increase: False
Jaccard    -> iters: 2 time(s): 0.6449848340016615 stop_centroid: False stop_sse_increase: True

```

Using the OR stopping rule (no centroid change **or** SSE increases **or** max iterations reached), Euclidean-K-means converged in **56 iterations** with total time about **12.45 s**, Cosine-K-means converged in **34 iterations** but took longer at about **15.17 s**, and Generalized Jaccard-K-means stopped after only **2 iterations** because the SSE increased, finishing in about **0.64 s**. Therefore, **Euclidean-K-means requires the most iterations** to converge, while **Cosine-K-means takes the most time overall** since cosine distance is more computationally expensive per iteration; Jaccard ends fastest due to early SSE-increase stopping.

In [14]: # Q4

In [15]: K = 10

```
def run_three_stops(X, dist_name, dist_func):
```



```
# 1) stop when no change in centroids
_, _, sse_centroid, it_centroid = kmeans(
    X, K, dist_func, max_iters=500, stop_rule="centroid"
)

# 2) stop when SSE increases next iteration
_, _, sse_sseinc, it_sseinc = kmeans(
    X, K, dist_func, max_iters=500, stop_rule="sse_increase"
)

# 3) stop when max preset iterations complete (use 100)
_, _, sse_max100, it_max100 = kmeans(
    X, K, dist_func, max_iters=100, stop_rule="max_only"
)

return {
    "Metric": dist_name,
    "SSE_stop_centroid": sse_centroid,
    "iters_stop_centroid": it_centroid,
    "SSE_stop_SSE_increase": sse_sseinc,
    "iters_stop_SSE_increase": it_sseinc,
    "SSE_stop_max100": sse_max100,
    "iters_stop_max100": it_max100
}

results = []
results.append(run_three_stops(X_eu, "Euclidean", euclidean_distance))
results.append(run_three_stops(X_cos, "Cosine", cosine_distance))
results.append(run_three_stops(X_jac, "Jaccard", jaccard_distance))

df_q4 = pd.DataFrame(results)
print(df_q4)
```

```

Converged in 56 iterations.
Reached max_iters=500.
Reached max_iters=100.
Converged in 34 iterations.
Reached max_iters=500.
Reached max_iters=100.
Converged in 55 iterations.
Converged in 2 iterations.
Reached max_iters=100.

```

	Metric	SSE_stop_centroid	iters_stop_centroid	SSE_stop_SSE_increase \
0	Euclidean	5.570521e+06	56	5.570521e+06
1	Cosine	6.864221e+02	34	6.864221e+02
2	Jaccard	3.665691e+03	55	4.035072e+03

	iters_stop_SSE_increase	SSE_stop_max100	iters_stop_max100
0	500	5.570521e+06	100
1	500	6.864221e+02	100
2	2	3.665691e+03	100

Under the three terminating conditions, Euclidean-K-means and Cosine-K-means behave consistently: stopping when centroids stop changing, stopping if SSE increases, or forcing a fixed maximum number of iterations all lead to essentially the same final SSE, because both methods converge smoothly and their SSE does not rise after convergence. In contrast, Jaccard-K-means is sensitive to the stopping rule: if we stop at the first SSE increase, it can terminate too early and yield a worse (higher) SSE, while stopping by centroid stability or allowing enough iterations (max-iters) lets it continue past that fluctuation and reach a lower, more stable SSE. Overall, centroid-no-change (and similarly max-iters) is the most reliable condition across methods, whereas the SSE-increase rule may prematurely stop Jaccard and degrade clustering quality.

In [17]: # Q5

Here are the main takeaways from the algorithmic analysis:

Overall, the choice of distance metric matters a lot for high-dimensional image-like data: cosine distance consistently produced the tightest clusters and the best label agreement, showing it captures the "shape/pattern" of the vectors better than raw magnitude-based Euclidean distance. Euclidean K-means needed the most iterations to settle, meaning it converged more slowly in this space, and its clustering quality (both SSE and accuracy) lagged

behind. Generalized Jaccard worked but was less stable: its SSE could temporarily increase early, so an “SSE-increase” stopping rule may stop too soon and hurt quality, whereas centroid-stability or enough iterations gives a better result. In short, cosine is the most suitable metric here, Euclidean is a weaker baseline for this dataset, and Jaccard is sensitive to stopping criteria, so convergence rules must be chosen carefully.